TÀI LIỆU TIẾNG VIỆT / 01

Sổ tay kiến thức

Hiểu paper từ nền tảng đến kiểm định và tái lập

Paper #366956 · EAI FISAT 2026

When Graph-Derived Neighbourhood Features Do Not Help: A Leakage-Controlled Evaluation of Behavioural-LRFM Churn Prediction in Non-Contractual Retail

Dành cho nhóm đã biết Python và machine learning cơ bản.
Phiên bản tài liệu: 13/10/2026. Nguồn: paper, code và bằng chứng tái lập.

Mục lục

1. Nghiên cứu này thực sự làm gì? 4

Quy ước bằng chứng . . . 5

2. Bản đồ toàn bộ paper và cách đọc 5

3. Churn không có hợp đồng: nhãn là một quyết định mô hình hóa 6

4. Từ giao dịch raw đến bảng khách hàng 6

4.1. Đơn vị quan sát và các bước làm sạch . . . 6

4.2. Cửa sổ thời gian và lỗi biên midnight . . . 7

4.3. Minh họa 1: một giao dịch ở biên . . . 7

5. Chín đặc trưng Behavioural-LRFM 7

5.1. L khác nhau giữa paper và code . . . 8

5.2. IPT và dữ liệu thiếu . . . 8

5.3. Minh họa 2: tự tính đủ chín biến . . . 8

5.4. Tương quan và dư thừa đặc trưng . . . 9

6. Chuẩn hóa: một phép đổi thang, hai vị trí trong pipeline 9

7. Phân khúc và dự đoán là hai mục tiêu khác nhau 10

7.1. KMeans và one-hot . . . 10

7.2. Maximum k-cut và quan hệ với bài trước . . . 10

8. Đồ thị khách hàng: kNN, hướng cạnh và trọng số 10

8.1. Tập kNN có hướng và graph union vô hướng . . . 11

8.2. k-sweep trả lời câu hỏi nào? . . . 11

9. Cộng đồng Louvain, Leiden và modularity 12

10. Bốn đặc trưng graph tĩnh 12

11. NCR và CCR: label aggregates cần ranh giới rõ 13

11.1. Minh họa 4: fold-local, fallback và nhãn của chính training row . . . 13

11.2. Ngữ nghĩa triển khai theo thời gian . . . 14

12. Các loại leakage và giao thức transductive 14

13. Mô hình học có giám sát và năm feature tiers 15

13.1. Logistic regression và gradient boosting . . . 15

13.2. Feature tiers là đối chứng nào? . . . 15

14. Repeated stratified paired cross-validation 16

15. ROC-AUC, PR-AUC và confusion matrix 16

Minh họa 5: cùng AUC, confusion matrix khác . . . 16

16. Mean, SD, CI và kiểm định: suy luận từ 50 fold 17

16.1. Mean và độ lệch chuẩn . . . 17

16.2. Bootstrap CI hiện tại . . . 17

16.3. Naive paired t và Nadeau-Bengio . . . 17

Minh họa 6: ghép cặp và correction . . . 18

16.4. Wilcoxon, multiple comparisons và null result . . . 18

17. Đọc đúng ba bảng kết quả 18

18. SHAP attribution khác predictive contribution 19

Minh họa 7: feature tương quan có SHAP mà không thêm ranking . . . 20

19. Silhouette và ưu tiên giữ chân 20

20. Tái lập, giới hạn và ý reviewer 20

21. Tự kiểm tra hiểu bài, kèm đáp án 21

22. Tài liệu nguồn và đường đọc tiếp 21

Thuật ngữ (Glossary) 23

Paper: **#366956** · Tên: *When Graph-Derived Neighbourhood Features Do Not Help: A Leakage-Controlled Evaluation of Behavioural-LRFM Churn Prediction in Non-Contractual Retail* · Nguồn: bản thảo TeX, PDF 12 trang, thư mục tham khảo · Người đọc: đã biết Python và học máy cơ bản · Ngày cập nhật: **2026-10-13**.

Tài liệu này giải thích kiến thức cần để hiểu và tự bảo vệ lập luận của paper. Nó được viết sau khi đọc mới toàn bộ 12 trang PDF, toàn bộ TeX và BibTeX, mã nguồn chính cùng bằng chứng chạy lại ngày 12/10. Các ví dụ nhỏ là dữ liệu tự tạo để học, không phải khách hàng thật hay kết quả thực nghiệm của nghiên cứu. Tài liệu giữ nguyên các artifact khoa học hiện có; những điểm chưa khớp được trình bày để người đọc biết giới hạn của bằng chứng.

1. Nghiên cứu này thực sự làm gì?

Chủ đề là dự đoán khách hàng ngừng mua trong bán lẻ không có hợp đồng bằng lịch sử giao dịch. **Bài toán cụ thể** là: tại một mốc phân tích, với mỗi khách đã mua trước đó, dự đoán họ có hoàn toàn không mua trong cửa sổ tương lai hay không. Đầu vào là dữ liệu mua hàng; đầu ra của mô hình là điểm hoặc xác suất cho nhãn bất hoạt. Không có trường dữ liệu ghi trực tiếp “khách đã rời bỏ doanh nghiệp”.

Mục tiêu nghiên cứu là kiểm tra liệu mở rộng RFM bằng thông tin hành vi, phân khúc và đặc trưng đồ thị có cải thiện khả năng phân biệt churn hay không. Ý tưởng ban đầu dễ hiểu: nếu những khách có hành vi giống nhau thường cùng ngừng mua, tỷ lệ churn trong vùng lân cận có thể bổ sung tín hiệu cho từng khách. Tuy nhiên, đồ thị ở đây được dựng từ chính chín đặc trưng hành vi; cạnh không ghi nhận khách gọi điện, giới thiệu nhau hay mua hàng cùng nhau. Đây là một giả thuyết dự báo về lân cận trong không gian đặc trưng, chưa phải chứng minh sự lan truyền xã hội.

Phương pháp gồm năm lớp đặc trưng tăng dần, hai bộ phân loại trong thí nghiệm chính, đồ thị kNN và cộng đồng Louvain, 10 lần lặp kiểm định chéo 5 fold, rồi so sánh hiệu số ghép cặp bằng kiểm định Nadeau-Bengio. Hai đặc trưng sử dụng nhãn, NCR và CCR, được tính lại từ nhãn tập train trong từng fold. Có thêm khảo sát bốn giá trị k để kiểm tra độ nhạy của kết quả với số láng giềng.

Đóng góp tăng thêm nằm ở kết quả âm có phạm vi và cách đánh giá cẩn thận: trên retailer, định nghĩa churn, họ đồ thị và cấu hình hiện tại, chưa phát hiện cải thiện có ý nghĩa của các tier so với RFM. Không nên diễn giải thành thuật toán đồ thị mới, SHAP mới, kiểm định mới hay bằng chứng mọi đồ thị bán lẻ đều vô ích. Phần nguyên lý SHAP khác đóng góp dự báo là bài học hữu ích, nhưng claim riêng “bỏ NCR không mất gì” còn thiếu ablation cô lập NCR.

Tiến độ hiện tại: các thực nghiệm đang có đã được chạy lại ngày 12/10/2026 trên Windows CPU, Python 3.13.5, dependency ghim. Mười công việc hoàn tất, tổng thời gian 17 phút 42 giây. Bộ k10 đọc cache mới khớp byte các fold scores và summary chính đã lưu; lượt đầu dùng DataFrame vừa dựng trong RAM có vài sai khác nhỏ. Những sai khác mô tả cửa sổ ngày, L, tập láng giềng, phạm vi SHAP và mức độ mạnh của kết luận chưa tự biến mất sau rerun. Chưa có bằng chứng fit lại mô hình với cửa sổ ngày trọn vẹn đã sửa. Báo cáo tái lập.

Những gì nhóm đang có gồm source trong src/, bản thảo/PDF, bảng điểm từng fold và summary trong results/, log chạy, audit, kết quả mới tách riêng trong notes/reproduce-20261012/, cùng notebook Colab và hướng dẫn. Notebook đã qua kiểm tra cấu trúc và helper nhưng chưa thực chạy toàn bộ trên Colab. Trạng thái “đã validate notebook” khác trạng thái “đã chạy Colab”.

Quy ước bằng chứng

Nhãn	Cách đọc
Paper mô tả	Nội dung bản thảo đang có
Code hiện tại	Hành vi đọc trực tiếp từ source
Kiểm chứng	Có artifact kiểm tra hoặc chạy thật
Minh họa	Số nhỏ tự tạo để giảng giải
Đề xuất chưa thực nghiệm	Hướng sửa hoặc thí nghiệm tương lai

Công thức được đánh dấu trong câu dẫn: **[paper]** là công thức paper nêu; **[derived]** là công thức suy diễn hoặc kiến thức nền để giải thích; **[illustration]** dùng cho ví dụ tự tạo. Không gán mọi công thức trong sổ tay cho tác giả paper.

2. Bản đồ toàn bộ paper và cách đọc

Paper có 11 phần đánh số, ngoài abstract và references. Thứ tự lập luận đi từ giả thuyết, cách tạo dữ liệu, kiểm soát đánh giá đến kết quả và giới hạn. Đọc phần kết quả mà bỏ qua cách tạo nhãn có thể khiến ta hiểu sai đối tượng mô hình đang dự đoán.

Phần paper	Ý chính	Kiến thức cần
Abstract, Introduction	Đặt giả thuyết và kết quả âm	Churn, baseline, leakage
Related Work	Biểu diễn, cấu trúc, đánh giá	RFM, đồ thị, kiểm định
Pipeline Overview	Dòng dữ liệu qua các bước	Feature/label separation
Data and Features	Cửa sổ, làm sạch, chín biến	Aggregation, timestamp
Graph and Communities	kNN, trọng số, Louvain/Leiden	Khoảng cách, modularity
Leakage Controls	Tính NCR/CCR trong train fold	Target encoding
Evaluation Protocol	Năm tier và 50 fold	Paired repeated CV
Results 8.1-8.5	Bảng 1-3, SHAP, phân khúc	AUC, CI, ablation
Discussion 9.1-9.2	Giải thích và giả thuyết tiếp	Null result, edge meaning
Threats to Validity	Giới hạn ngoại suy	Dataset, horizon, validation
Conclusion và cuối bài	Phạm vi kết luận, tái lập, AI	Provenance, trách nhiệm

Paper mô tả: ba dòng nghiên cứu liên quan gặp nhau. Dòng biểu diễn mở rộng điều đo về khách, như tenure, demographic hay RFM theo thời gian. Dòng cấu trúc xem khách là nút trong đồ thị. Dòng đánh giá hỏi liệu chênh lệch rất nhỏ có vượt nhiễu do chia fold hay không. Nghiên cứu này dùng công cụ có sẵn ở hai dòng đầu và nhấn mạnh dòng thứ ba.

Cần giữ đúng quan hệ với bài segmentation của Corrêa Vianna Filho và cộng sự. Bài đó đặt phân khúc RFM thành bài toán maximum k-cut; nghiên cứu hiện tại thêm bài toán supervised churn. Mở rộng phạm vi bài toán không đồng nghĩa cải thiện chất lượng phân cụm hay vượt phương pháp cũ trên cùng benchmark. Nguồn bài segmentation.

3. Churn không có hợp đồng: nhãn là một quyết định mô hình hóa

Rời bỏ có hợp đồng (contractual churn) thường có một sự kiện quan sát: chấm dứt thuê bao, hủy gói hoặc không gia hạn hợp đồng. Rời bỏ không có hợp đồng (non-contractual churn) không cung cấp tín hiệu rõ như vậy. Khách có thể im lặng trong nhiều tháng rồi mua lại. Trong dữ liệu này, “churn” phải được vận hành hóa thành không xuất hiện giao dịch hợp lệ trong một khoảng tương lai đã chọn.

Code hiện tại: nhãn bằng 1 nếu CustomerID ở observation không thuộc tập khách mua trong prediction; ngược lại bằng 0. Nhãn không nói khách đã chuyển sang đối thủ, không hài lòng hoặc vĩnh viễn rời đi. Vì vậy tên sát nghĩa là “bất hoạt theo horizon” (inactivity label). Đây không chỉ là việc chọn từ: nó quyết định loại câu hỏi nghiên cứu mà các phép đo có thể trả lời.

Một người thường mua hàng mỗi 180 ngày có thể bị gắn churn theo horizon ba tháng dù vẫn là khách trung thành. Người mua rất đều mỗi tuần có thể có một lần mua nhỏ trong prediction và được gắn không churn dù doanh thu giảm mạnh. Cả hai tình huống đều phù hợp với code. Mô hình tốt cho nhãn này chưa chắc tốt cho xác suất rời bỏ lâu dài, mất doanh thu, lifetime value hoặc hiệu quả giữ chân.

Minh họa: khách A mua ngày 01/06 và 01/10, không mua trong quý III. Với nhãn bất hoạt quý III, A là churner. Khi đánh giá hồi cứu tới tháng 10, ta biết A quay lại; điều đó không làm nhãn quý III sai theo định nghĩa, nhưng cho thấy không nên diễn giải nhãn thành rời bỏ vĩnh viễn. Nếu thay horizon, mẫu dữ liệu có thể giữ gần như nguyên mà nhãn đổi.

Churn rate **65,2%** không phải bằng chứng phần lớn khách thất vọng. Nó là tỷ lệ không có lần mua hợp lệ theo cửa sổ hiện tại. Có **3.285** positives trên **5.038** khách; positive là lớp đa số. Mô hình luôn dự đoán churn có accuracy khoảng 65,2%, nhưng gần như không có năng lực xếp hạng. Khi trình bày, phải nói rõ `1=churn`, bởi đổi positive class sẽ đổi precision/recall/AP và cách hiểu class weighting.

Phân biệt hợp đồng với không hợp đồng không đồng nghĩa mọi bài telecom đều có nhãn hợp đồng hoàn toàn giống nhau. Khi dùng kết quả telecom làm nền, phải xem định nghĩa nhãn cụ thể. Điểm khác biệt chắc chắn trong nghiên cứu này là cạnh similarity tính từ feature matrix khác cạnh giao tiếp quan sát. Bài TempODEGraphNet dùng tương tác người chơi và đồ thị động; kết quả của nó không phải benchmark trực tiếp cho retail kNN này. Nguồn TempODEGraphNet.

4. Từ giao dịch raw đến bảng khách hàng

4.1. Đơn vị quan sát và các bước làm sạch

Raw Online Retail II là transaction log, không phải một bảng 5.038 dòng sẵn có. Hai worksheet có **525.461** và **541.910** dòng, tổng **1.067.371**. Một invoice thường có nhiều dòng sản phẩm, nên đếm dòng khác đếm hóa đơn và khác đếm khách. Mô hình cuối cùng dự đoán ở cấp khách hàng, sau phép nhóm `groupby("CustomerID")`. UCI Online Retail II.

Code hiện tại: đọc cả hai worksheet, bỏ khoảng trắng trong tên cột, chuyển InvoiceDate sang datetime và loại dòng thiếu CustomerID. Dấu invoice bắt đầu bằng `C` được dùng để nhận diện hủy. Các dòng hủy được tách trước khi lọc giao dịch mua; nếu lọc Quantity dương trước, thông tin hủy có thể mất. Dữ liệu mua chính giữ invoice không hủy, Quantity > 0, Price > 0, rồi bỏ dòng trùng. Tiền dòng được tính `TotalPrice = Quantity * Price`.

Đây là định nghĩa doanh thu từ các giao dịch mua hợp lệ, không phải lợi nhuận hay doanh thu ròng sau hoàn trả. Hủy được dùng như một biến riêng; code không ghép từng invoice hủy về hóa đơn gốc để trừ khỏi M. Tránh gọi M là “giá trị kinh tế thật của khách” nếu chưa bổ sung discount, margin, hoàn tiền và chi phí phục vụ.

Ví dụ một invoice có ba dòng sản phẩm được tính một lần vào Frequency. Hai dòng cùng StockCode trong một invoice không làm số loại sản phẩm của BasketSize tăng gấp đôi. Các phép nunique, sum, mean trả lời những câu hỏi khác nhau; nhóm cần nhận biết chúng khi đọc source. Dòng trùng bị loại trong bảng mua chính, nhưng CancellationRate đếm số invoice hủy phân biệt nên số dòng hủy cũng không trực tiếp là tử số của tỷ lệ.

4.2. Cửa sổ thời gian và lỗi biên midnight

Paper mô tả: observation 01/12/2009-30/06/2011; prediction 01/07/2011-30/09/2011. Các đặc trưng chỉ dùng observation, nhãn dùng prediction. Về ý tưởng, đây là ngăn thông tin tương lai đi vào đặc trưng của thời điểm dự đoán.

Code hiện tại: OBS_END = pd.Timestamp("2011-06-30") và PRED_END = pd.Timestamp("2011-09-30") đều là 00:00:00. Điều kiện observation là InvoiceDate <= OBS_END; prediction là InvoiceDate > OBS_END và <= PRED_END. Giao dịch lúc 10:00 ngày 30/06 rơi vào prediction. Giao dịch lúc 10:00 ngày 30/09 không nằm trong prediction. Điều kiện không có mốc bắt đầu tường minh; dữ liệu thực tế bắt đầu tháng 12/2009 nên phần đầu theo nguồn dữ liệu.

Kiểm chứng: audit đếm **1.186** dòng mua hợp lệ ban ngày 30/06 bị chuyển khỏi observation và **1.451** dòng ban ngày 30/09 bị loại khỏi prediction. Nếu dùng ngày trọn vẹn như lời mô tả, observation có **557.398** dòng và **5.041** khách; **28 nhãn** đổi ở khách chung. Đây là kiểm đếm dữ liệu, chưa phải kết quả model được fit lại. Audit thực nghiệm.

Đề xuất chưa thực nghiệm: dùng khoảng nửa mở (half-open interval) để biểu đạt ngày trọn vẹn: observation trước 01/07/2011 00:00; prediction từ mốc đó đến trước 01/10/2011 00:00. Quý III có 92 ngày; gọi nó "90 ngày" chỉ là mô tả gần đúng và nên sửa nếu muốn chính xác. Nếu đổi code, phải xây lại cache, graph và model rồi đánh giá lại; không dùng các số đếm mới để thay bảng AUC cũ.

4.3. Minh họa 1: một giao dịch ở biên

Giả sử khách B chỉ mua lúc 30/06/2011 15:00 và 30/09/2011 12:00. Theo code hiện tại, B không có giao dịch observation nên không trở thành một dòng trong working matrix; lần đầu bị đưa vào prediction và lần sau bị bỏ khỏi prediction. Theo cửa sổ ngày trọn vẹn, B là khách observation và có mua prediction, nên nhãn bằng 0. Sai khác biên có thể đổi cả việc khách có được đưa vào nghiên cứu và nhãn của họ, không chỉ đổi một chữ số thời gian.

5. Chín đặc trưng Behavioural-LRFM

RFM là ba biến tóm tắt: recency R cho biết lần mua cuối đã cách bao lâu; frequency F cho biết số hóa đơn; monetary M cho biết tổng tiền. LRFM thêm relationship length L. Behavioural-LRFM trong repo thêm AOV, BasketSize, ProductDiversity, CancellationRate và InterpurchaseTime. Những biến này cùng lấy từ lịch sử mua, nhưng nhấn mạnh thời gian, cường độ, quy mô, độ rộng và hủy giao dịch.

Gọi t_a là ngày phân tích, t_1 và t_{last} là timestamp đầu và cuối trong observation, F_i là số invoice mua hợp lệ của khách i, C_i là số invoice hủy trong observation. Công thức dưới đây là **[derived]**, diễn đạt hành vi code; không phải tất cả đều được viết thành phương trình trong paper.

Biến	Định nghĩa code	Đơn vị/ý nghĩa
L	$t_a - t_1$ lấy số ngày nguyên	Tuổi quan hệ tới mốc phân tích
R	$t_a - t_{\text{last}}$ lấy số ngày nguyên	Độ lâu kể từ lần mua cuối
F	Invoice mua phân biệt	Số lần đặt hàng

Biến	Định nghĩa code	Đơn vị/ý nghĩa
M	Tổng Quantity × Price	Tổng tiền mua hợp lệ
AOV	M_i/F_i	Tiền trung bình mỗi invoice
BasketSize	Mean số StockCode/invoice	Độ rộng giỏ hàng
ProductDiversity	StockCode phân biệt của khách	Độ rộng danh mục mua
CancellationRate	$C_i/(C_i + F_i)$	Tỷ lệ hóa đơn hủy
InterpurchaseTime	Mean khoảng cách invoice	Số ngày giữa lần mua

5.1. L khác nhau giữa paper và code

Paper mô tả: L là số ngày giữa lần mua đầu và cuối. **Code hiện tại:** L là ngày phân tích trừ lần mua đầu. Nếu dùng cùng đơn vị liên tục, hai cách liên hệ bằng công thức **[derived]**:

$$L_{\text{code}} = (t_{\text{last}} - t_1) + R$$

Vì code lấy .dt.days và IPT cũng chuyển chênh timestamp thành ngày nguyên, đẳng thức ngày nguyên có thể chênh một đơn vị do phần giờ. Ý nghĩa vẫn rõ: L code chứa thêm thời gian bất hoạt từ lần mua cuối tới ngày phân tích. Khách chỉ mua một lần có L paper bằng 0 nhưng L code có thể rất lớn. Hai biến không thể đổi tên cho nhau mà coi là không đổi phương pháp. Source tạo đặc trưng.

5.2. IPT và dữ liệu thiếu

Khoảng cách giữa hai lần mua (interpurchase time, IPT) phải tính ở cấp invoice, vì nhiều sản phẩm cùng hóa đơn không phải nhiều lần mua. Code lấy timestamp đầu mỗi invoice, sắp thứ tự, lấy khoảng cách rồi lấy trung bình. Với $F_i \geq 2$, dạng công thức **[derived]** là:

$$\text{IPT}_i = \frac{1}{F_i - 1} \sum_{r=1}^{F_i - 1} (t_{r+1} - t_r)$$

Khách có một invoice không đủ khoảng cách để tính IPT; gán 0 sẽ mang ý nghĩa giả rằng mua gần như liên tục. Code gán NaN rồi điền median IPT của nhóm có nhiều invoice. Median ấy được tính trên toàn working population trước CV, nên đây là xử lý covariates transductive. Nếu muốn kiểm tra generalisation cho khách mới theo giao thức inductive, median cũng nên được fit ở train. Đó là đề xuất riêng, chưa phải thực nghiệm đã có.

5.3. Minh họa 2: tự tính đủ chín biến

Tất cả số sau đây là **Minh họa**. Dùng ngày phân tích 01/07/2011 và các lần mua đúng 00:00 để tránh làm tròn giờ. Khách C mua ba invoice: 01/01 giá 30 với hai loại sản phẩm; 01/03 giá 40 với ba loại; 01/06 giá 50 với một loại. Tổng cộng có bốn StockCode phân biệt; một invoice hủy nằm trong observation.

Ta có $F = 3$, $M = 120$, AOV = 40, BasketSize = $(2 + 3 + 1)/3 = 2$, ProductDiversity = 4, CancellationRate = $1/(1 + 3) = 0,25$. Từ 01/01 đến 01/07 là 181 ngày nên $L_{\text{code}} = 181$; từ 01/06 đến 01/07 là 30 ngày nên $R = 30$. Khoảng đầu-cuối là 151 ngày, đúng bằng $181 - 30$ trong ví dụ này. IPT = $(59 + 92)/2 = 75,5$ ngày.

Nếu khách C mua lại 15/08, nhãn bằng 0; nếu không mua trong prediction, nhãn bằng 1. Chín feature vẫn giống nhau trong hai tình huống giả định, vì chúng được tính từ quá khứ. Ta không được dùng lần mua tháng 8 để cập nhật Frequency hoặc M rồi vẫn gọi đây là dự đoán tại đầu tháng 7.

Nếu một khách D chỉ có một invoice 01/06, $L_{\text{code}} = R = 30$, $F = 1$, AOV bằng M, IPT cần điền median; L first-to-last bằng 0. Trường hợp này giúp nhớ rằng không phải biến nào cũng mang cùng lượng thông tin cho khách ít lịch sử.

5.4. Tương quan và dư thừa đặc trưng

Có nhiều biến hơn không đồng nghĩa có nhiều thông tin độc lập hơn. AOV được tính từ M và F; tuổi quan hệ code, recency và span đầu-cuối có liên hệ; IPT có quan hệ với span và số hóa đơn. Với timestamp liên tục, tổng các khoảng liên tiếp bằng $t_{\text{last}} - t_1$, nên IPT bằng span chia $F - 1$. Sau làm tròn mỗi gap thành ngày nguyên, quan hệ có thể không còn khớp tuyệt đối nhưng vẫn mạnh.

Trong ML, một feature xác định từ các feature khác đôi khi vẫn có ích về mặt biểu diễn. Ví dụ logistic regression tuyến tính trên M, F không tự tính được tỷ số M/F bằng một tổ hợp tuyến tính; thêm AOV có thể giúp lớp mô hình ấy. Ngược lại, một model phi tuyến có thể đã học gần đủ quan hệ, nên biến mới đem lại rất ít. “Dư thừa theo dữ liệu” và “dư thừa đối với một model hữu hạn” là hai mức khác nhau.

Nếu khoảng cách Euclidean dùng nhiều biến tương quan, một khía cạnh hành vi có thể được tính trọng lượng nhiều lần. Hai feature cùng phản ánh tần suất mua khiến nhóm “mua đều” chiếm nhiều chiều. StandardScaler cân thang đo từng cột, không làm các cột độc lập. Đây là lý do cần đánh giá cả khoảng cách, biểu diễn và feature tier, thay vì mặc định graph chín chiều luôn sâu hơn RFM ba chiều.

6. Chuẩn hóa: một phép đổi thang, hai vị trí trong pipeline

Chuẩn hóa z-score (standardisation) đưa mỗi cột về thang tương đối so với phân phối dùng để fit scaler. Công thức nền **[derived]**:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

μ_j là mean và σ_j là độ lệch chuẩn cột j trong tập fit. Scaler lưu hai đại lượng này để transform dữ liệu sau. Biến sau chuẩn hóa không mặc nhiên có phân phối Gaussian; đuôi dài của Monetary không biến mất chỉ bằng trừ mean và chia SD. StandardScaler cũng nhạy với outlier. Tài liệu StandardScaler.

Minh họa: hai khách chênh R 10 ngày, F 2 invoice, M 1.000 đơn vị tiền. Khoảng cách raw chủ yếu do M; mô hình graph dễ chọn láng giềng theo tiền. Nếu SD của ba cột lần lượt là 20, 4, 2.000, cả ba chênh lệch chuẩn hóa bằng 0,5, và mỗi chiều đóng góp 0,25 vào bình phương khoảng cách. Đây là lựa chọn hình học: “chênh một SD” được coi ngang nhau giữa các cột. Nó không chứng minh mức chênh ấy ngang nhau về ý nghĩa kinh doanh.

Code hiện tại có hai scaler khác nhau. Scaler dựng graph/KMeans được fit trên toàn bộ chín feature của 5.038 khách trước khi chia CV. Scaler đứng trong Pipeline của classifier được fit chỉ trên train của mỗi fold, rồi transform validation. Phát biểu “mọi standardisation đều fold-local” sai với source; phát biểu “classifier scaler fold-local” đúng. Phân biệt này là điều cần nhớ khi đọc phần Leakage Controls.

Đối với logistic regression có regularisation, scale thay đổi mức phạt tương đối lên hệ số, nên chuẩn hóa có ý nghĩa rõ. Với cây quyết định, phép đổi thang tuyến tính đơn điệu giữ thứ tự cột và thường không tạo lợi ích tương tự; code vẫn đưa mọi classifier qua cùng scaler. Không suy ra scaler giúp LightGBM chỉ vì nó nằm trong pipeline. Đối với kNN và KMeans, scale trực tiếp thay đổi distance, neighbor list và partition.

Một phép biến đổi fit từ validation covariates cần được nêu phạm vi. Trong giao thức transductive, toàn bộ covariates của quần thể đích có thể được biết trước; trong giao thức inductive cho khách chưa xuất hiện, việc sử dụng chúng trong fit không mô phỏng đúng triển khai. Dù không dùng nhãn, thay đổi phân phối validation

vẫn có thể đổi scaler và đồ thị. Đánh giá cần quyết định câu hỏi nào đang được trả lời trước khi chọn preprocessing.

7. Phân khúc và dự đoán là hai mục tiêu khác nhau

Phân cụm không giám sát (unsupervised clustering) chia nhóm theo cấu trúc X, không học từ nhãn churn y. Học có giám sát (supervised learning) dùng cặp (X, y) để học một hàm dự đoán. Một partition có vẻ rõ ràng theo hành vi vẫn có thể không bổ sung AUC, nếu những feature cá nhân đã chứa thông tin cần để dự đoán nhãn.

7.1. KMeans và one-hot

KMeans tìm tâm cụm sao cho tổng bình phương khoảng cách trong cụm nhỏ. Dạng mục tiêu nền **[derived]**:

$$J = \sum_{i=1}^{N} \|z_i - \mu_{c_i}\|^2$$

Ở đây c_i là cluster của khách, μ_{c_i} là tâm cụm, khác μ_j của scaler. **Code hiện tại:** KMeans 5 cụm, random_state=42, n_init=10 trên chín feature đã chuẩn hóa. Số cụm KMeans 5 không phải số láng giềng kNN 10; hai chữ k trong cùng nghiên cứu chỉ hai tham số khác nhau. Tài liệu KMeans.

Cluster ID là nhãn định danh. Mã 4 không cao gấp hai lần mã 2 và không mang thứ tự “loyal đến churn”. Nếu đổi tên cụm, partition vẫn như cũ. Đưa số nguyên đó vào logistic regression sẽ áp một cấu trúc tuyến tính tùy tiện; đưa vào tree có thể tạo thứ tự chia tùy tiện. One-hot biến mỗi nhóm thành cột chỉ báo, để model có thể học hiệu ứng riêng của nhóm.

Minh họa: ba cụm A/B/C được mã 0/1/2. Khi bỏ cột đầu, A biểu diễn [0,0], B [1,0], C [0,1]. Ta không mất nhóm A; nó trở thành nhóm tham chiếu. KMeans có 5 nhóm nên code sinh 4 dummy; Louvain có 20 nhóm nên sinh 19 dummy; tổng 23. drop_first=True tránh phụ thuộc tuyến tính hoàn toàn với intercept. Đây không phải phép rút gọn học từ nhãn.

7.2. Maximum k-cut và quan hệ với bài trước

Maximum k-cut tìm partition để tối đa hóa tổng trọng số các cạnh nối hai nhóm khác nhau. Nếu trọng số biểu thị khác biệt, cách đặt mục tiêu này có thể giúp tách những khách khác nhau. KMeans tối thiểu hóa distance tới tâm; Louvain tìm cộng đồng gắn kết theo modularity. Các phương pháp đang tối ưu các đại lượng khác nhau, nên không thể đọc mọi “cluster score” như cùng một nhiệm vụ.

Paper mô tả: bài trước dùng RFM score rời rạc 1-5 và gom những vector score giống nhau thành super-vertices. Điều này có thể giảm bài toán xuống số kiểu score thay vì số khách; trong ba chiều mỗi chiều tối đa 5 mức, số kiểu tối đa 125. Đây là suy diễn đếm tổ hợp, không phải khẳng định mọi dataset thực sự có đủ 125 kiểu. Nghiên cứu hiện tại dùng chín chiều liên tục để dự đoán churn, không triển khai supervised maximum k-cut classifier. Nguồn bài trước.

8. Đồ thị khách hàng: kNN, hướng cạnh và trọng số

Đồ thị (graph) là tập nút và cạnh. Một nút đại diện một khách; cạnh cần có ý nghĩa cụ thể. Trong repo, cạnh nói hai khách gần nhau trong không gian Behavioural-LRFM đã chuẩn hóa. Nó không nói họ có tương tác thật. Chọn distance và scale là chọn định nghĩa “giống nhau”, vì thế graph là một sản phẩm của thiết kế biểu diễn.

Khoảng cách Euclidean nền **[derived]**:

$$d(i,j) = \sqrt{\sum_{r=1}^{9} (z_{ir} - z_{jr})^2}$$

Code hiện tại: NearestNeighbors(n_neighbors=k+1, metric="euclidean"); truy vấn cả working matrix; bỏ vị trí đầu được kỳ vọng là chính khách; lưu k láng giềng. Với dữ liệu hiện tại, fresh check không tìm self-neighbour. Ở $k = 10$, mỗi neighbor_dict[i] có 10 lựa chọn. Trọng số **[paper]** là:

$$w(i,j) = \frac{1}{1 + d(i,j)}$$

Hai khách càng gần thì cạnh càng mạnh, trọng số nằm trong $(0, 1]$. Nếu distance 0 thì weight 1; distance 1 thì weight 0,5; distance 3 thì weight 0,25. Hàm này không phải xác suất churn chung, độ tin cậy nhân quả hay correlation. Nó chỉ là phép đổi distance thành mức tương tự để thuật toán đồ thị sử dụng.

8.1. Tập kNN có hướng và graph union vô hướng

“B gần nhất của A” không đảm bảo “A gần nhất của B”. Nếu quanh B có nhiều khách gần hơn, quan hệ kNN có hướng. Code giữ danh sách lựa chọn này để tính NCR, nhưng thêm cạnh vào nx.Graph() vô hướng: có cạnh nếu ít nhất một đầu chọn đầu kia. Đây là đối xứng hóa bằng union (union symmetrisation), khác mutual kNN chỉ giữ khi cả hai chọn nhau.

Minh họa 3: trên một trục đã chuẩn hóa, bốn điểm A=0, B=1, C=1,1, D=3, dùng k=1. A chọn B; B chọn C; C chọn B; D chọn C. Graph union có cạnh A-B, B-C, C-D. Degree của B và C bằng 2, dù mỗi outgoing list có một phần tử. NCR của B theo code chỉ dùng C; NCR theo toàn graph neighbor sẽ dùng A và C. Hai cách không tương đương.

Kiểm chứng: graph k10 có **5.038** nút, **33.426** cạnh. Audit cho degree union từ **10 đến 25**, trung bình khoảng **13,26955**, và **4.498** nút có degree > 10. Vì vậy cụm từ “fixed degree 10” chỉ phù hợp cho độ dài outgoing list, không cho degree của graph vô hướng. Paper nên dùng hai ký hiệu riêng: $\mathcal{N}_{\text{out}}(i)$ cho kNN list và $\mathcal{N}_G(i)$ cho graph neighbors. Audit và fresh kNN check.

Đây là sự khác biệt có thể kiểm tra trực tiếp trong loop tạo graph. neighbor_dict[i].append(j) chạy cho từng lựa chọn; G.add_edge(i,j) chỉ thêm nếu cạnh chưa có. Số lựa chọn có hướng là $Nk = 50.380$ ở k10; số cạnh union thấp hơn do các lựa chọn đôi được gộp. Không nhân hay chia máy móc để suy ra chính xác 33.426 nếu chưa biết số cặp chọn lẫn nhau.

8.2. k-sweep trả lời câu hỏi nào?

Tăng k mở rộng vùng địa phương. Nhóm gần có thể giữ tính tương tự tốt nhưng NCR có ít labels train để lấy mean; nhóm rộng có nhiều labels hơn nhưng có thể trộn hành vi khác. Đây là đánh đổi bias/variance hợp lý của phép làm trơn. Vì vậy sign của hiệu số có thể thay đổi theo k ngay cả khi tồn tại một effect thật. Việc sign đổi không tự chứng minh “chỉ là nhiễu”.

Kiểm chứng: $k = 5, 10, 15, 20$ đều đã được chạy lại. Không setting nào có Graph-vs-RFM LightGBM $p_{\text{NB}} < 0,05$. Kết quả tăng sức tin rằng kết luận hiện tại không phụ thuộc duy nhất k10, nhưng vẫn thuộc cùng một họ graph, cùng distance, scale và weight. Nó chưa kiểm tra referral, household, co-purchase, GNN hay một metric learned khác.

9. Cộng đồng Louvain, Leiden và modularity

Cộng đồng (community) là nhóm có kết nối nội bộ mạnh so với mức tham chiếu của graph. Nó không nhất thiết là cụm hình cầu trong feature space. Louvain bắt đầu từ các nhóm nhỏ, chuyển nút để tăng modularity rồi gom cộng đồng thành super-nodes, lặp lại. Đây là tối ưu heuristic, phụ thuộc graph, weight và lựa chọn ngẫu nhiên. Bài Louvain.

Một công thức modularity cho graph vô hướng có trọng số, **[derived]** theo nền Newman-Girvan:

$$Q = \frac{1}{2m}\sum_{i,j}\left(A_{ij} - \frac{s_i s_j}{2m}\right) I(c_i = c_j)$$

A_{ij} là weight cạnh, s_i là tổng weight quanh nút, $2m = \sum_i s_i$, và $I(c_i = c_j)$ bằng 1 khi cùng cộng đồng. Số hạng $s_i s_j/(2m)$ là mức kết nối kỳ vọng trong null model giữ strength. Q lớn nghĩa có nhiều weight nằm trong cộng đồng hơn kỳ vọng đó; không phải 83,85% dự đoán churn chính xác. Nguồn modularity.

Code hiện tại: Louvain tối ưu có weight, seed 42. Leiden chạy ModularityVertexPartition, n_iterations=10, seed 42 trong pipeline v2. Leiden thêm bước refinement để tránh những cộng đồng kém kết nối mà Louvain có thể tạo. Bảo đảm của Leiden là bảo đảm cấu trúc theo thuật toán, không phải bảo đảm feature community cải thiện AUC. Bài Leiden.

Kiểm chứng: pipeline hiện tại tạo Louvain 20 cộng đồng, modularity **0,8385**; Leiden 20 cộng đồng, **0,8323**. Hai số không dẫn tới câu “Louvain là model churn tốt hơn Leiden”, bởi repeated prediction experiment chỉ dùng dummy và CCR của Louvain. Source đã chạy partition Leiden nhưng chưa tạo một đối chiếu prediction lặp có cùng controls thay riêng detector. Một auxiliary variant có Leiden còn tạo label aggregates toàn cục, nên không thay thế phép thử đó.

High modularity cần được diễn giải theo graph construction. kNN ưu tiên cạnh cục bộ trong space đã tạo; graph có thể có modularity cao vì cách chọn cạnh. Một graph phân vùng rõ vẫn có thể chia khách theo kích thước giỏ hàng hay monetary mà target churn không hưởng thêm thông tin. Muốn nói cấu trúc phản ánh nhóm khách thực, cần ground truth hoặc external validation độc lập; modularity tự nó chỉ đánh giá partition trên graph.

10. Bốn đặc trưng graph tĩnh

Tĩnh ở đây nghĩa chúng chỉ dùng cấu trúc graph từ covariates, không dùng nhãn churn. “Tĩnh” không có nghĩa bất biến nếu raw data, scaler, k, weight hay customer population đổi. Trong main CV chúng được dựng một lần trước folds.

Đặc trưng	Nó đo gì?	Nó không chứng minh gì?
Degree	Số graph neighbors union	Số bạn bè thật
WeightedDeg	Tổng weight quanh nút	Giá trị kinh tế của khách
PageRank	Điểm từ dòng truyền trên graph	Quyền lực xã hội thực
ClustCoeff	Mức nối nhau của hàng xóm	Xác suất churn

Degree **[derived]** là $D_i = |\mathcal{N}_G(i)|$; weighted degree là $s_i = \sum_{j \in \mathcal{N}_G(i)} w_{ij}$. Với ba cạnh weight 0,8; 0,6; 0,2, degree bằng 3 nhưng strength bằng 1,6. Hai khách cùng degree có thể có neighborhood xa/gần khác nhau. Degree ở đây tính trên union, còn NCR dùng outgoing list.

PageRank có trực giác một random walker chuyển theo weight, thỉnh thoảng quay về một nút theo phân phối nền. Dạng nền **[derived]**:

$$p_i = \frac{1-\alpha}{N} + \alpha \sum_j \frac{w_{ji}}{S_j} p_j$$

Với graph vô hướng như hiện tại, NetworkX dùng hai hướng tương ứng để tính. Code không truyền alpha nên dùng mặc định thư viện. PageRank ở similarity graph thường phản ánh việc một khách nằm ở vùng kết nối mạnh, không phải vị trí trong mạng giao tiếp. Tài liệu PageRank.

Clustering coefficient không trọng số có trực giác $2T_i/[D_i(D_i-1)]$, trong đó T_i là số tam giác qua nút. Nếu ba hàng xóm có đủ ba cạnh nối nhau thì hệ số bằng 1; chỉ có một cạnh thì bằng 1/3. **Code hiện tại dùng phiên bản weighted của NetworkX**, dựa vào mean hình học của weight tam giác đã chuẩn hóa, nên không được dùng công thức unweighted để tính lại chính xác CSV. Tài liệu weighted clustering.

11. NCR và CCR: label aggregates cần ranh giới rõ

Tỷ lệ churn láng giềng (Neighbour Churn Rate, NCR) và tỷ lệ churn cộng đồng (Community Churn Rate, CCR) là đặc trưng có nhãn. Chúng khác Degree/PageRank vì không tính được chỉ từ covariates. Nếu đổi labels, NCR/CCR đổi dù graph giữ nguyên.

Gọi $\mathcal{T}$ là index train của một fold, $\bar{y}_{\mathcal{T}}$ là mean label train. Đối với NCR, dùng $A_i = \mathcal{N}_{\text{out}}(i) \cap \mathcal{T}$. Nếu A_i không rỗng, công thức **[derived, theo code]** là:

$$\text{NCR}_i = \frac{1}{|A_i|} \sum_{j \in A_i} y_j$$

Nếu rỗng, code dùng $\bar{y}_{\mathcal{T}}$. Trung bình này **không có trọng số distance**, dù graph có weights. NCR không phải trung bình weighted theo w_{ij}. Paper nêu phép giao train-fold nhưng gọi tập neighbors theo graph; source cần làm rõ outgoing kNN list.

Với cộng đồng c_i, đặt $B_i = \{j \in \mathcal{T} : c_j = c_i\}$. Khi không rỗng, **[derived, theo code]**:

$$\text{CCR}_i = \frac{1}{|B_i|} \sum_{j \in B_i} y_j$$

Khi cộng đồng không có ai trong train, dùng mean train. Mọi khách cùng cộng đồng nhận cùng CCR trong một fold. Với validation row, nhãn row đó không nằm trong mean. Với training row, nhãn chính row đó hiện **có nằm** trong mean cộng đồng. Hai mức kiểm soát này cần tách nhau.

11.1. Minh họa 4: fold-local, fallback và nhãn của chính training row

Tạo tám khách A-H, labels lần lượt 1,0,1,0,1,0,1,0. Train là A,B,C,D; validation là E,F,G,H; mean train bằng 0,5. Ví dụ dùng ba outgoing neighbors mỗi khách để dễ tính. E có outgoing neighbors A,C,F. NCR(E) chỉ lấy A,C nên bằng $(1+1)/2 = 1$. Label F=0 không được dùng. Nếu tính full-data NCR, kết quả là 2/3; chỉ đổi label F cũng làm feature của E đổi dù F đang validation.

F có outgoing neighbors E,G,H, đều đã được xác định là validation, nên không có train neighbor. NCR(F) trở về 0,5; không gán 0, vì "chưa có thông tin" khác "mọi láng giềng đều không churn". Fallback phản ánh mức nguy cơ nền train, tránh mean trên tập rỗng.

Giả sử cộng đồng E là {A,B,E}. CCR(E) = $(1+0)/2 = 0,5$. Khi tính CCR(A) để fit model, source cũng cho 0,5, gồm chính label A=1. Nếu dùng leave-one-out, CCR(A) chỉ dùng B và bằng 0; CCR(B) chỉ dùng A và bằng 1.

Ví dụ nhóm hai train members cho thấy self-inclusion có thể tạo khác biệt mạnh giữa feature train và feature validation. Với nhóm lớn, mức tác động trực tiếp của một nhãn giảm, nhưng không tự bằng 0.

Một cộng đồng chỉ có E,F, không có ai train, sẽ nhận CCR bằng 0,5 cho cả hai. Trường hợp cộng đồng có duy nhất A ở train thì code cho CCR(A)=1; validation members nhóm đó cũng nhận 1. Train row lúc này có một feature chứa trọn label của chính nó. Đây là lý do target encoding thường dùng cross-fitting khi tạo feature training. Tài liệu TargetEncoder về internal cross-fitting.

Kiểm chứng: main code chặn validation labels trong NCR/CCR của mỗi fold. **Đề xuất chưa thực nghiệm:** leave-one-out hoặc inner cross-fitting cho training CCR; cân nhắc smoothing về mean train để nhóm nhỏ ổn định hơn. Khi đổi cách tính training features, cần fit lại và so cùng folds; chưa có kết quả chứng minh biến thể này tốt hơn.

11.2. Ngữ nghĩa triển khai theo thời gian

Trong offline CV, labels train là dữ liệu đã biết để học. Tuy nhiên, labels đang là bất hoạt trong chính prediction window của quần thể này. Nếu muốn dùng hệ thống thật vào đầu tháng 7 để dự đoán quý III, ta chưa biết quý III churn của các khách “train neighbors” tại thời điểm đó. Giao thức hiện tại đo dự đoán labels held-out khi có labels của phần còn lại trong một snapshot lịch sử; nó chưa tự chứng minh pipeline prospective có đủ đầu vào lúc bắt đầu horizon.

Đề xuất triển khai cần nói NCR lấy từ nhãn lịch sử đã hoàn tất horizon trước đó, hay từ tín hiệu lân cận hiện thời nào. Đây là suy luận về tính sẵn có dữ liệu, không phải kết luận rằng code đã đọc validation labels. Một đánh giá rolling theo thời gian với feature availability rõ sẽ kiểm tra câu hỏi triển khai khác. Nhóm nên trình bày NCR như label-derived feature trong giao thức đã định, rồi giới hạn ứng dụng tương lai tương ứng.

12. Các loại leakage và giao thức transductive

Rò rỉ dữ liệu (data leakage) xảy ra khi thông tin ngoài phạm vi được phép đi vào quá trình học hoặc chọn model, khiến phép đánh giá trả lời câu hỏi dễ hơn triển khai thực. Cần chỉ rõ đường đi thông tin, thay vì gọi mọi vấn đề là “leak”. Hướng dẫn common pitfalls.

Loại	Ví dụ trong bài toán này	Trạng thái
Temporal leakage	Hủy sau observation vào feature	Main đã giới hạn thời gian
Validation-target leakage	NCR dùng label validation	Main đã chặn theo fold
Covariate preprocessing	Fit graph scaler trên tất cả X	Có; phạm vi transductive
Train target encoding	CCR train chứa chính y của row	Có; chưa cross-fit riêng
Selection leakage	Chọn k/model theo test rồi báo test	Cần quản lý giao thức

Một đối chiếu hữu ích là “ai biết gì, vào thời điểm nào”. Main CV dùng toàn bộ observation covariates để dựng graph, KMeans và communities, sau đó chỉ dùng train labels để tạo aggregates. Đây là transductive covariates kết hợp fold-local labels. Nó khác việc preprocessing và graph đều học chỉ trên train rồi gắn khách mới vào graph khi inference, tức inductive evaluation.

Full-data SHAP không tự làm main CV leak. Source tạo mô hình báo cáo sau khi CV hoàn tất. Việc mô hình báo cáo dùng toàn labels không quay ngược thay đổi điểm CV nếu điểm được tạo từ pipeline riêng đúng controls. Nhưng SHAP và các giá trị ưu tiên in-sample đó phải được mô tả theo đúng nguồn gốc; không được gọi chúng là held-out explanation hoặc campaign validation.

13. Mô hình học có giám sát và năm feature tiers

13.1. Logistic regression và gradient boosting

Logistic regression học một điểm tuyến tính rồi đổi qua sigmoid. Công thức nền **[derived]**:

$$f(x) = b + \sum_j \beta_j x_j, \qquad p(x) = \frac{1}{1 + e^{-f(x)}}$$

Nó tuyến tính trong các feature được đưa vào, nhưng xác suất không tuyến tính trong score. Thêm AOV hay dummy có thể mở rộng lớp hàm so với chỉ ba biến RFM. Regularisation ℓ_2 phạt hệ số lớn để cân khả năng fit và độ ổn định. **Code hiện tại:** LogisticRegression(max_iter=1000, random_state=42, class_weight="balanced"); các mặc định còn lại theo scikit-learn ghim. max_iter=1000 là trần bước solver, không nghĩa chắc chắn đã học 1.000 epochs. Tài liệu LogisticRegression.

LightGBM là ensemble cây boosted: cây tiếp theo sửa sai theo loss của ensemble đang có. Dạng biểu diễn nền **[derived]** là $f_M(x) = f_0(x) + \eta \sum_{m=1}^{M} h_m(x)$. **Code hiện tại:** 300 estimators, learning rate 0,05, max depth 6, seed 42, is_unbalance=True. n_estimators=300 chỉ số cây boosted được yêu cầu, không phải 300 lượt đi qua dữ liệu kiểu training neural network. Main không truyền callback early stopping; không nên tự thêm câu “chọn best epoch” vào mô tả. Tài liệu LightGBM.

Bản pipeline một partition còn chạy Decision Tree, Random Forest và XGBoost. Tree tạo các luật chia theo feature; Random Forest kết hợp nhiều cây qua bagging; XGBoost cũng là gradient boosting. Tuy nhiên, kết luận repeated-CV chính dựa trên **LightGBM và logistic regression**, không phải cả năm classifier đều đã nhận cùng kiểm định lặp trong manuscript. Script auxiliary chứa voting ensemble/PCA và biến thể khác; không gán chúng thành components của thí nghiệm chính.

Class weighting thay đổi ảnh hưởng từng lớp lên loss, không tạo thêm thông tin. Với balanced weights, công thức nền là $w_c = N/(2N_c)$. Theo tỷ lệ churn 65,2%, positive weight khoảng 0,77 và negative weight khoảng 1,44; lớp không churn mới là lớp ít hơn. Đây là ví dụ cho thấy “churn hiếm nên phải oversample” không đúng ở dataset này. Cân trọng số cũng có thể làm probability calibration khác tần suất thật; AUC tốt không đủ xác nhận điểm dự đoán là xác suất kinh doanh đã hiệu chuẩn.

13.2. Feature tiers là đối chứng nào?

Tier	Thành phần	Số cột k10
RFM	R, F, M	3
LRFM	RFM + L code	4
B-LRFM	Chín biến hành vi	9
B-LRFM+Seg	B-LRFM + 23 dummy	32
Graph-Enhanced	Seg + 4 tĩnh + NCR + CCR	38

Giữ classifier và folds cố định, so các tier trả lời hiệu quả tăng thêm của **block** đặc trưng. RFM→LRFM cô lập việc thêm L; LRFM→B-LRFM thêm năm biến cùng lúc; Seg→Graph thêm sáu biến cùng lúc. Vì vậy bảng này không phải ablation riêng từng graph feature. Static-only có trong single-partition pipeline nhưng không phải một tier repeated-CV chính. Nếu đổi cả classifier và tier, phép so chỉ còn là so hai pipelines tổng thể.

14. Repeated stratified paired cross-validation

Kiểm định chéo 5 fold chia khách thành năm nhóm. Mỗi lượt fit dùng bốn nhóm, validation ở nhóm còn lại; mỗi khách có một prediction held-out trong một repetition. Stratified giữ tỷ lệ labels trong các fold gần tỷ lệ chung. Nó giúp fold ít chênh class balance, nhưng không tạo các thử nghiệm độc lập hay thay thế hold-out theo thời gian. Tài liệu StratifiedKFold.

Code hiện tại: mười seeds chia fold mặc định là 1000-1009; seed graph/model là 42. Trong mỗi repetition, tất cả tier và model dùng cùng cặp train/validation indices. Có 50 fold scores cho mỗi cấu hình; năm tier × hai classifier cho **500 rows** mỗi bộ. Một khách được validation mười lần trên các partitions khác nhau; chưa có 50 dataset hay 50 nhóm khách độc lập.

Ghép cặp (paired) quan trọng vì độ khó fold ảnh hưởng đồng thời hai cấu hình. Nếu model A đánh giá ở nhóm khách dễ hơn model B, chênh score vừa chứa model effect vừa chứa partition effect. Cùng folds cho phép tính $d_r = s_{A,r} - s_{B,r}$ trên cùng khách; nhờ đó phần độ khó chung có thể triệt bớt. Vẫn phải hiệu chỉnh dependence do training sets overlap.

“Single split” trong paper là **một lần chia thành 5 folds**, không phải một train/test hold-out đơn lẻ. Mean 5-fold vẫn chỉ dùng một partition seed. Mười repetitions kiểm tra độ nhạy với partition, nhưng đều thuộc một observation/prediction snapshot. Không được gọi repeated random CV này là temporal backtesting qua mười quý.

Các hyperparameters đang cố định, không có nested CV tuning trong main. Nếu sau này chọn model/$_k$ bằng chính score đã báo, cần một vòng chọn cấu hình và một đánh giá ngoài độc lập. **Đề xuất chưa thực nghiệm:** rolling origin theo thời gian và outer hold-out cho quyết định triển khai; đó là câu hỏi khác với tái lập suite hiện tại.

15. ROC-AUC, PR-AUC và confusion matrix

Khả năng phân biệt (discrimination) là xếp churners cao hơn non-churners. ROC-AUC dùng toàn bộ ranking qua các thresholds, trong khi F1/recall tính ở một threshold. Công thức nền **[derived]**: TPR = TP/(TP+FN), FPR = FP/(FP+TN), precision = TP/(TP+FP), recall = TPR, F1 = $2PR/(P+R)$. Không thể dùng duy nhất một confusion matrix để suy ra toàn đường ROC.

ROC-AUC có thể đọc như tỷ lệ cặp positive-negative được xếp đúng, thêm nửa điểm cho tie. Dạng **[derived]**:

$$\text{AUC} = \frac{1}{N_+ N_-} \sum_{i:\, y_i = 1} \sum_{j:\, y_j = 0} \left[I(s_i > s_j) + \tfrac{1}{2} I(s_i = s_j) \right]$$

0,5 là mức ranking ngẫu nhiên trong kỳ vọng; 1 là ranking hoàn hảo trên dữ liệu đánh giá. AUC 0,81 không nghĩa 81% khách được dự đoán đúng ở threshold 0,5, cũng không nghĩa xác suất một khách cụ thể churn là 0,81. Code dùng predict_proba(...)[:,1] với roc_auc_score. Tài liệu ROC-AUC.

Minh họa 5: cùng AUC, confusion matrix khác

Tạo sáu khách xếp theo score giảm dần: (0,9;1), (0,8;0), (0,7;1), (0,4;0), (0,3;1), (0,1;0). Mỗi cặp là (score,label). Có chín cặp positive-negative: positive 0,9 thắng 3, positive 0,7 thắng 2, positive 0,3 thắng 1. AUC = 6/9 = 0,667.

Ở threshold 0,5: TP=2, FP=1, FN=1, TN=2; precision=recall=F1=2/3. Ở threshold 0,2: TP=3, FP=2, FN=0, TN=1; precision=0,6 và recall=1. AUC không đổi vì ranking chưa đổi. Do đó chọn threshold là quyết định dựa trên chi phí/capacity khác với lựa chọn model theo AUC.

PR curve dùng precision và recall. **Code hiện tại gọi average_precision_score, tức average precision (AP), dù cột và paper dùng tên PR-AUC.** AP tính tổng precision theo lượng recall tăng, không phải diện tích trapezoid với nội suy tuyến tính. Nền **[derived]**:

$$\mathrm{AP} = \sum_r (\mathrm{Recall}_r - \mathrm{Recall}_{r-1})\mathrm{Precision}_r$$

Trong ví dụ tự tạo, positives xuất hiện ở rank 1,3,5, nên AP = $(1 + 2/3 + 3/5)/3 = 0,756$. Nó khác ROC-AUC vì hỏi mức tập trung positives ở đầu ranking. Tài liệu average precision.

Khi positives chiếm khoảng 65,2%, precision nền cũng đã cao. AP 0,87 phải được đọc so với prevalence này, không được mang sang dataset có churn 5% như cùng độ khó. AP vẫn hữu ích; chỉ là không mang cách diễn giải “phát hiện rare churner” thông thường. Ngay cả khi ROC-AUC và AP đều tăng trên một partition, chúng vẫn dùng cùng khách và không trở thành hai bằng chứng độc lập.

16. Mean, SD, CI và kiểm định: suy luận từ 50 fold

16.1. Mean và độ lệch chuẩn

Mean ROC-AUC là trung bình 50 score. SD mô tả độ phân tán của score giữa folds, không phải khoảng tin cậy của mean. Source dùng SD mẫu với ddof=1. Với hiệu số ghép cặp $d_1, \ldots, d_n$, các công thức **[derived]** là:

$$\bar{d} = \frac{1}{n}\sum_{r=1}^{n} d_r, \qquad s_d^2 = \frac{1}{n-1}\sum_{r=1}^{n}(d_r - \bar{d})^2$$

SD của score riêng và SD của paired differences có thể rất khác. Nếu cả hai model cùng lên/xuống ở một fold, absolute score SD lớn nhưng difference SD nhỏ. Figure 2 có bands ±1 SD của fold scores; bands chồng nhau không phải phép kiểm định paired differences. Cũng không nên đọc band ấy như CI95%.

16.2. Bootstrap CI hiện tại

Source lấy 10.000 mẫu bootstrap từ vector 50 differences, có hoàn lại; tính mean từng mẫu và lấy percentile 2,5/97,5, seed 42. Đây là một cách mô tả effect magnitude dưới giả định bootstrap đang dùng. Folds lặp share dữ liệu train, nên resample từng difference như độc lập không bảo toàn đầy đủ dependence; CI có thể hẹp hơn mức bất định thích hợp.

Vì thế CI bootstrap không chứa 0 và $p_{\mathrm{NB}} > 0,05$ không phải mâu thuẫn logic: hai phép tính dùng cách xử lý dependence khác. Khoảng 95% cũng không có nghĩa “95% xác suất effect thật nằm ở đây” theo diễn giải frequentist. Paper đã cảnh báo không dùng CI bootstrap này như kết luận significance; nhóm cần giữ cảnh báo khi rút bảng ra làm slide.

16.3. Naive paired t và Nadeau-Bengio

Naive paired t dùng SE = $s_d/\sqrt{n}$, giả định các difference có dependence phù hợp mô hình độc lập. Các train sets của 5-fold overlap rất nhiều; lặp thêm không tạo 50 lần học độc lập. Kiểm định corrected resampled t tăng variance. Công thức **[paper]**:

$$t_{\mathrm{NB}} = \frac{\bar{d}}{\sqrt{s_d^2\left(\frac{1}{n} + \frac{n_{\mathrm{test}}}{n_{\mathrm{train}}}\right)}}$$

Ở 5-fold, tỷ số test/train xấp xỉ 1/4; với $n = 50$, hệ số variance bằng 0,27 thay vì 0,02. Code dùng t-distribution df=49 và p hai phía. Correction là một phép xấp xỉ suy luận có giả định, không tạo độc lập thật; nhưng tránh sự tự tin quá mức của naive test trong thiết kế này. Nguồn Nadeau-Bengio.

Minh họa 6: ghép cặp và correction

Tạo bốn folds với score RFM [0,80;0,82;0,79;0,81] và Graph [0,803;0,821;0,789;0,815]. Differences là [0,003;0,001;−0,001;0,005], mean 0,002, variance mẫu khoảng 0,000006667. Naive SE khoảng 0,001291. Nếu minh họa vẫn dùng test/train=1/4, corrected SE bằng $\sqrt{0,000006667(1/4 + 1/4)} \approx 0,001826$. Statistic giảm từ khoảng 1,549 xuống 1,095. Đây là ví dụ bốn fold scores tự tạo, không phải một cấu hình chạy thật.

Với ví dụ khác gồm 50 differences có mean 0,003 và SD 0,01, naive SE là 0,001414 và t≈2,12; corrected SE là 0,005196 và t≈0,577. Tăng độ tin cậy bằng đếm nhiều folds độc lập giả có thể đảo kết luận. Điều quan trọng là variance của **paired differences**, tỷ lệ train/test và thiết kế lấy mẫu, không chỉ số score có trong CSV.

16.4. Wilcoxon, multiple comparisons và null result

Wilcoxon signed-rank dùng dấu và thứ hạng độ lớn differences; nó giảm lệ thuộc giả định Gaussian so với t-test nhưng vẫn có giả định về distribution/dependence. “Nonparametric” không có nghĩa không cần độc lập, không bị overlap hoặc luôn đúng. Source báo Wilcoxon như cross-check; paper lấy p_{NB} làm phép suy luận chính. Tài liệu Wilcoxon.

Có 16 phép so tier-vs-RFM từ bốn tier thêm × hai model × hai metric, ngoài cross-model và k-sweep. Mỗi p được tính ở mức từng phép thử; chưa có correction cho cả họ so sánh trong báo cáo. Nếu 16 tests độc lập cùng null và mỗi test mức 0,05, xác suất có ít nhất một false positive là $1 - 0,95^{16} \approx 0,56$; **Minh họa** này không là xác suất thực tế của các tests phụ thuộc trong paper. Nó giải thích vì sao cần nói rõ primary comparisons, exploratory comparisons và multiple-testing policy. Bài Demšar về so sánh classifier.

$p > 0,05$ nghĩa chưa đủ bằng chứng bác bỏ null trong phép kiểm định, không nghĩa effect bằng 0, không nghĩa xác suất null đúng lớn hơn 95%. Muốn kết luận tương đương (equivalence), phải chọn trước một biên effect nhỏ nhất đáng quan tâm, rồi dùng giao thức và khoảng bất định phù hợp để kiểm hai phía trong biên. Chưa có equivalence test hoặc power study ở suite này. Một effect nhỏ có thể có ý nghĩa vận hành ở quy mô lớn; “significant” và “đáng triển khai” cũng là hai câu hỏi khác nhau.

17. Đọc đúng ba bảng kết quả

Các bảng dưới chép số làm tròn **Paper mô tả**, đã được đối chiếu với archived results và cached k10 rerun. Dùng báo cáo tái lập khi cần số nhiều chữ số hoặc nhánh raw/RAM khác.

Tier LightGBM	Seed42 AUC	Repeated AUC	Delta vs RFM	pNB
RFM	0,8065	0,8066	-	-
LRFM	0,8071	0,8074	+0,0009	0,755
B-LRFM	0,8067	0,8054	−0,0012	0,742
B-LRFM+Seg	0,8073	0,8044	−0,0022	0,561
Graph-Enhanced	0,8098	0,8044	−0,0021	0,590

Bảng 1 hỗ trợ câu “chưa phát hiện cải thiện của tier so với RFM cùng classifier”. Mean gần nhau không chứng minh bằng nhau. Graph tốt nhất ở một partition nhưng delta âm qua mười repetitions; vì thế không chọn headline +0,0033 mà bỏ qua repeated evaluation. Cả 16 tests tier-vs-RFM đều p≥0,05, nhỏ nhất khoảng 0,144.

Cặp pipeline, A trừ B	Delta AUC	Bootstrap CI95%	pNB
LR/B-LRFM − LGBM/Graph	+0,0113	[+0,0096; +0,0130]	0,001
LR/RFM − LGBM/Graph	+0,0099	[+0,0080; +0,0119]	0,010
LR/RFM − LGBM/RFM	+0,0078	[+0,0056; +0,0100]	0,068

Bảng 2 có **hai** hàng p<0,05 chưa điều chỉnh multiple comparisons, không phải chỉ một significant comparison. Hai hàng đầu đổi cả model và features; không chứng minh riêng graph làm giảm AUC hoặc capacity bị tiêu vào variance. Hàng thứ ba đổi riêng classifier ở RFM nhưng p≈0,068. “Logistic regression pipeline thắng graph pipeline trong các cặp này” là câu sát bằng chứng hơn.

kNN k	Edges	Louvain groups	Delta Graph−RFM	pNB
5	16.902	31	−0,0006	0,880
10	33.426	20	−0,0021	0,590
15	50.054	16	+0,0017	0,666
20	66.652	16	−0,0027	0,500

Bảng 3 kiểm tra độ nhạy trong họ kNN này. Nó không kiểm tất cả graph families. Discussion gọi khoảng 0,005 AUC là noise floor; nên đọc là heuristic cảnh giác trên dữ liệu này. Chưa có power/equivalence study để biến nó thành ngưỡng phát hiện đã chứng minh, cũng chưa có controlled experiment chỉ đổi library version để xác định nguyên nhân drift.

18. SHAP attribution khác predictive contribution

SHAP phân bổ phần chênh đầu ra model so với baseline cho các feature. Biểu diễn cộng nền **[derived]**:

$$f(x) = \phi_0 + \sum_j \phi_j(x)$$

ϕ_j giải thích model đã fit, không là causal effect của can thiệp vào feature. Mean absolute SHAP là trung bình $|\phi_j|$; bỏ dấu nên không nói feature luôn tăng nguy cơ. Bài SHAP.

Code hiện tại: sau CV, tạo NCR/CCR từ toàn labels, fit model full-data, lấy tối đa 800 training samples để giải thích, rồi xuất global và per-community SHAP. TreeExplainer mặc định giải thích raw output; với LightGBM binary là margin/log-odds, không là phần trăm xác suất. Tài liệu TreeExplainer.

Top năm mean absolute SHAP được paper báo là R **1,087**, M **0,560**, NCR **0,250**, IPT **0,224**, CCR **0,215**. Các số tái lập được; phạm vi là mô tả model full-data này. Chúng không tự là held-out attributions của 50 CV models. Phân tích mô tả full-data hợp lệ nếu ghi rõ phạm vi; nó không tự làm main CV metric leak.

Minh họa 7: feature tương quan có SHAP mà không thêm ranking

Tạo hai feature trùng nhau $x_2 = x_1$ trên toàn population, centered baseline 0. Model toy đã fit là $f = x_1 + x_2$; với khách có $x_1 = x_2 = 2$, raw score bằng 4. Trong phân bổ đối xứng toy, mỗi feature nhận 2 đơn vị attribution. Bỏ x_2 rồi **fit lại** model $g = 2x_1$ cho cùng score và cùng ranking. Attribution trước khi bỏ vẫn cao; predictive contribution thêm của feature thứ hai bằng 0 trong ví dụ nhân tạo này. Đây là suy diễn minh họa, không là TreeSHAP đã tính trên retail.

Không fit lại mà thay x_2 bằng 0 hỏi một câu khác: model hiện tại phản ứng thế nào khi thiếu đầu vào. Khi features chỉ tương quan gần, attribution còn phụ thuộc cách SHAP xử lý dependence và background. Vì thế SHAP rank cao có thể phù hợp với feature redundancy, nhưng không chứng minh redundancy đã là nguyên nhân trong thí nghiệm thật.

Thí nghiệm loại bỏ thành phần (ablation) phải thay đúng thành phần, fit lại và giữ controls. Main Seg→Graph thêm bốn static features và hai aggregates cùng lúc. Fresh paired Graph−Seg LightGBM delta khoảng **+0,00005118**, pNB≈**0,98693**; corrected interval khoảng **[−0,00620; +0,00630]**. Nó hỗ trợ chưa thấy gain của block sáu biến, không chứng minh NCR-only cost=0. **Đề xuất chưa thực nghiệm:** cùng folds cho static-only, NCR-only, CCR-only và các combinations; không dùng feature importance để thay ablation.

19. Silhouette và ưu tiên giữ chân

Silhouette đo sự gắn kết theo distance: với a_i là mean distance trong nhóm, b_i là mean distance tới nhóm khác gần nhất, công thức nền **[derived]** là $s_i = (b_i - a_i)/\max(a_i, b_i)$. Gần 1 nghĩa tách tốt theo metric ấy; âm nghĩa có thể gần nhóm khác hơn. Nó không dùng nhãn churn. Tài liệu silhouette.

Khác feature space, scaling, metric, window hoặc population thì score không tạo benchmark đối đầu công bằng. Paper ghi baseline RFM-score ba chiều **0,398**, KMeans chín chiều **0,219**, Louvain **−0,050**. **Kiểm chứng:** số 0,398 là published reference trong code comparison; output local approximation không chứng minh câu “we reproduce 0,398”. Cần bổ sung provenance hoặc thu hẹp câu đó. Comparison script dùng toàn transaction history, còn churn dùng observation; cần nêu riêng window khi dùng các silhouette numbers.

Code hiện tại: ExpectedRetention = ChurnProbability * M. Đây là score ưu tiên “rủi ro × giá trị quá khứ”, chưa phải expected incremental retention revenue. Nó thiếu xác suất campaign thay đổi hành vi, margin và chi phí. High-value at-risk 327 khách/82% churn, high-value loyal 418 khách/24% churn là mô tả các nhóm trong artifact, không là chiến dịch đã giữ lại khách.

Minh họa: khách nguy cơ 0,9, monetary 100 nhận score 90; khách nguy cơ 0,3, monetary 1.000 nhận 300. Khách thứ hai được ưu tiên cao hơn theo quy tắc, nhưng chưa chắc đáp ứng campaign. **Đề xuất chưa thực nghiệm:** randomized campaign với control, đo response/lift/lợi ích tăng thêm; dùng held-out ranking và probability calibration trước quyết định ngân sách. Dataset không có campaign-response labels để xác nhận những claim đó.

20. Tái lập, giới hạn và ý reviewer

Tái lập cần data hash, source commit, dependency, seeds, code path, output và log. Seed không đủ khi environment hoặc parser khác. Cache tiết kiệm đọc XLSX nhưng có thể stale; source hiện tại reuse theo tồn tại file, chưa tự fingerprint raw/window. Đợt kiểm chứng đã xây cache mới từ raw, nên có bằng chứng mạnh hơn chỉ đọc cache cũ.

Kiểm chứng ngày 12/10: 43 tracked files của bộ kiểm tra lúc ấy khớp remote, raw local và raw tải UCI cùng SHA256; 10 jobs exit 0, Windows CPU/Python 3.13.5, 17 phút 42 giây. Cached k10 mới khớp main archived;

raw/RAM đầu khác nhẹ, nguyên nhân riêng parser/precision chưa được controlled test. Notebook Colab đã validate, chưa thực chạy Colab. Không dùng thời gian local làm ETA đã đo cho Colab. Run manifest, comparison manifest.

Mặt tái lập	Mức	Bằng chứng/giới hạn
Environment	full cho local rerun	Runtime/dependency ghim
Data	partial về mô tả	Hash/count có; date prose lệch
Training	substantial	Source chạy; nhiều defaults
Evaluation	substantial	Paired CV/NB; transductive scope
Ablation causality	partial	Block test; thiếu NCR-only
Code availability	full cho suite hiện hữu	Log/output mới; không mọi proposal

Phán quyết: **suite hiện hữu tái lập được**, nhưng phương pháp mô tả và các claim chưa hoàn toàn khớp implementation. Một dataset, một retailer, một snapshot và một họ graph giới hạn ngoại suy. Country encoding và spectral clustering được paper ghi chưa implemented; k-sweep đã completed, không còn là component thiếu theo notes cũ. Literature search là targeted, không systematic; không đủ priority/“first” claim.

Reviewer yêu cầu làm rõ non-contractual inactivity, ý nghĩa similarity edges từ cùng features, giới hạn ngoại suy sang interaction graphs, và việc Table 2 đổi cả classifier/features. Nên dùng Table 1 cùng classifier làm bằng chứng feature-tier effect. Các ý này phù hợp với ranh giới trong sổ tay; được ghi lại trong báo cáo review. Review #3 không đưa yêu cầu kỹ thuật cụ thể. Paper còn khai báo dùng generative AI để hỗ trợ viết/code/đối chiếu; trách nhiệm kiểm chứng vẫn thuộc tác giả.

21. Tự kiểm tra hiểu bài, kèm đáp án

1. **AUC 0,81 có nghĩa accuracy 81% không?** Không; đó là ranking metric, accuracy cần threshold.
2. **L code có bằng first-to-last không?** Không; nó đo từ first tới analysis date, chứa thêm recency.
3. **Graph degree luôn bằng k không?** Không; outgoing list dài k, union degree có thể lớn hơn.
4. **NCR dùng weights không?** Main lấy mean labels train không trọng số trên outgoing list.
5. **CCR train đã loại chính label row chưa?** Chưa; chặn validation labels khác cross-fit training encoding.
6. **SHAP NCR đứng thứ ba chứng minh tăng AUC không?** Không; attribution và contribution là hai câu hỏi.
7. **pNB 0,590 chứng minh effect 0 không?** Không; đây là failure to reject, chưa equivalence.
8. **So LR/RFM với LGBM/Graph cô lập graph không?** Không; cả model và features đều đổi.
9. **Đã rerun toàn suite có nghĩa sửa date-window đã được tested không?** Không; rerun giữ source, window sửa chưa fit.
10. **Có notebook nghĩa đã chạy Colab không?** Không; cần runtime log và outputs của lần chạy Colab thật.

22. Tài liệu nguồn và đường đọc tiếp

Nguồn quyết định hành vi là main.tex, main.pdf, run_pipeline_v2.py, significance_test.py, compare_methods_v2.py và improvements_pipeline.py. Nguồn trạng thái mới là report/manifest ngày 12/10; audit trước đó chỉ được kế thừa cho findings phương pháp còn chưa sửa. Official docs được mở ngày 13/10 giải thích khái niệm; phiên bản docs hiện hành không được coi là runtime đã chạy.

BibTeX/PDF có **17** tài liệu tham khảo. Bản đồ ngắn sau giữ đủ các entry; không tuyên bố đã deep-read toàn văn mọi bài liên quan hay khảo sát literature có hệ thống.

Nguồn	Vai trò	Đường dẫn
Vianna Filho et al., 2026	Graph RFM segmentation	Bài
Blondel et al., 2008	Louvain	Bài
Traag et al., 2019	Leiden	Bài
Newman-Girvan, 2004	Modularity	Bài
Chen-Guestrin, 2016	XGBoost	Bài
Ke et al., 2017	LightGBM	Bài
Lundberg-Lee, 2017	SHAP	Bài
Chen/UCI, 2019	Dataset	UCI
Nadeau-Bengio, 2003	Corrected test	Bài
Dietterich, 1998	Tests for algorithms	Bài
Bouckaert-Frank, 2004	Test replicability	Bài
Demšar, 2006	Multiple-dataset comparisons	JMLR
Chen et al., 2012	Retail RFM	Bài
Mena et al., 2023	Time-varying RFM	Bài
Ho et al., 2023	Extended RFM	Bài
Kostić et al., 2020	Telecom social graph	Bài
Lee-Woo, 2025	Dynamic game graph	Bài

Thuật ngữ (Glossary)

English	Tiếng Việt	Giải thích ngắn
Churn	Rời bỏ/bất hoạt	Phải đọc theo định nghĩa label
Horizon	Khoảng dự đoán	Thời gian dùng tạo outcome
RFM/LRFM	Bộ tóm tắt lịch sử	R/F/M và thêm L
Covariate	Biến đầu vào	Thông tin X
Target	Nhãn đích	Outcome y
Standardisation	Chuẩn hóa	Đổi scale theo mean/SD
Redundancy	Dư thừa	Thông tin có thể đã nằm ở biến khác
kNN	Láng giềng gần nhất	Chọn theo distance
Symmetrisation	Đối xứng hóa	Đổi quan hệ có hướng thành cạnh
Community	Cộng đồng	Partition theo graph
Modularity	Độ module	So nội bộ với graph null model
Degree/strength	Bậc/tổng trọng số	Đếm cạnh/cộng weight
NCR/CCR	Tỷ lệ churn lân cận	Aggregates của nhãn
Target encoding	Mã hóa từ nhãn	Feature dùng outcome train
Cross-fitting	Fit chéo nội bộ	Tạo train feature từ folds khác
Transductive	Suy luận trên quần thể đã biết X	Có covariates đích khi dựng representation
Inductive	Suy luận cho đối tượng mới	Fit representation trên train
Paired CV	CV ghép cặp	Các model dùng cùng partitions
Discrimination	Khả năng phân biệt	Xếp hạng positive/negative
Calibration	Hiệu chuẩn xác suất	Probability khớp tần suất quan sát
AP	Average precision	Tổng precision theo tăng recall
SD/SE	Độ lệch chuẩn/sai số chuẩn	Spread của quan sát/bất định estimator
CI	Khoảng tin cậy	Phải đọc theo phép xây khoảng
Null result	Kết quả chưa bác bỏ null	Chưa đủ evidence khác biệt
Equivalence	Tương đương	Effect nằm trong biên định trước
Attribution	Phân bổ giải thích	Vai trò trong model đã fit
Ablation	Loại bỏ thành phần	Fit lại để kiểm phần tăng thêm
Uplift	Tác động tăng thêm	Outcome với treatment trừ control
Provenance	Nguồn gốc bằng chứng	Data/code/run tạo con số